

LAPORAN PRAKTIKUM
MATA KULIAH BIG DATA FUNDAMENTAL
Eksplorasi Data Film Populer dari API TMDb



Dosen Pengampu : Ainul Yaqin, S ,Kom., M.Kom.

Disusun Oleh:

Nama Kelompok : Erdhien Nizar Tharegi (23.01.5076)
Azrilla Dimas Putra Tama (23.01.5078)
Muhammad Rizki Fajri (23.01.5094)

PROGRAM STUDI D3 TEKNIK INFORMATIKA
FAKULTAS ILMU KOMPUTER
UNIVERSITAS AMIKOM YOGYAKARTA

2025

PENDAHULUAN

Dalam era digital saat ini, industri perfilman mengalami pertumbuhan yang sangat pesat dengan ribuan judul film baru dirilis setiap tahunnya di berbagai belahan dunia. Data film bukan hanya digunakan oleh penonton untuk mencari rekomendasi tontonan, tetapi juga menjadi sumber informasi penting bagi para pelaku industri hiburan, peneliti tren pasar, hingga pengembang teknologi rekomendasi film. Salah satu sumber data terbuka yang paling populer adalah The Movie Database (TMDb), sebuah platform berbasis komunitas yang menyediakan berbagai informasi mendetail tentang film, serial TV, rating, popularitas, hingga ulasan penonton.

Dalam konteks pembelajaran Big Data Fundamental, data film dari TMDb dapat dijadikan studi kasus yang relevan untuk mempraktikkan bagaimana data mentah dikumpulkan secara langsung melalui API publik, diolah menjadi dataset terstruktur, lalu divisualisasikan dalam bentuk dashboard interaktif. Melalui studi kasus ini, mahasiswa diharapkan dapat memahami bagaimana alur kerja pengolahan data dilakukan secara end-to-end, mulai dari proses crawling data film populer, penyimpanan data ke dalam format CSV, pembersihan data agar lebih konsisten dan mudah dianalisis, hingga pembuatan visualisasi untuk melihat pola-pola tertentu seperti tren rata-rata popularitas film atau rating film dari tahun ke tahun.

Dengan studi kasus ini, mahasiswa juga dapat melihat secara langsung bagaimana data yang awalnya hanya berupa angka mentah dapat diubah menjadi informasi yang bermanfaat untuk mendukung pengambilan keputusan. Misalnya, dengan menganalisis popularitas dan rating film dari berbagai tahun, kita dapat menemukan kecenderungan genre film yang diminati, periode perilisian dengan antusiasme penonton tertinggi, atau bagaimana penilaian rata-rata publik berubah dari waktu ke waktu. Penerapan proyek sederhana seperti ini juga menumbuhkan kesadaran akan pentingnya data pipeline yang rapi, mulai dari proses pengambilan data (data ingestion), penyimpanan (storage), pembersihan (preprocessing), hingga penyajian (presentation) yang informatif.

Secara keseluruhan, studi kasus ini dirancang tidak hanya sebagai latihan teknis pengolahan data, tetapi juga sebagai jembatan bagi mahasiswa untuk memahami bagaimana keterampilan Big Data dapat diimplementasikan pada data nyata dan memiliki relevansi tinggi di berbagai bidang industri.

METODOLOGI

Secara garis besar, tahap pengerjaan tugas ini terbagi menjadi beberapa langkah, yaitu:

1. Crawling Data (Raw Data)

Data diambil melalui API endpoint `/movie/popular` dari TMDb. Proses dilakukan untuk beberapa halaman dengan parameter `page` agar data lebih bervariasi. Hasilnya berupa data mentah dalam format JSON yang berisi atribut-atribut seperti judul film, popularitas, rating, dan tanggal rilis.

2. Penyimpanan ke CSV

Data mentah kemudian dikonversi menjadi DataFrame menggunakan pustaka `pandas` dan disimpan ke dalam file CSV (`tmdb_movies.csv`). Penyimpanan ini bertujuan untuk memastikan data dapat diakses ulang tanpa harus melakukan *request* API berulang kali.

3. DataFrame dan Preprocessing

Pada tahap ini, data dibersihkan dengan cara:

- Mengubah kolom `release_date` menjadi format tanggal yang valid.
- Mengekstrak tahun rilis ke kolom `year`.
- Mengonversi kolom `popularity` dan `vote_average` ke numerik.
- Memilih kolom yang relevan untuk dianalisis lebih lanjut.

Hasil preprocessing ini memastikan data konsisten dan bebas dari *null value* yang dapat mengganggu analisis.

4. Visualisasi Interaktif dengan Dash

Data yang telah diproses divisualisasikan dalam bentuk dashboard interaktif menggunakan pustaka Dash. Fitur yang dibuat meliputi:

- Tabel daftar film populer dengan pagination.
- Pilihan metrik yang dapat diubah secara interaktif, yaitu `popularity` atau `vote_average`.
- Grafik batang yang menampilkan rata-rata metrik terpilih per tahun rilis.

Dengan adanya visualisasi ini, pola tren dapat diamati dengan lebih mudah.

TOOLS

- Google Collab
- Python
- dash untuk dashboard interaktif
- Plotly Express untuk visualisasi grafik
- Pandas untuk pengolahan data
- Requests untuk mengakses API TMDb

IMPLEMENTASI PROGRAM

INSTALL LIBRARY

```
pip install dash
Requirement already satisfied: dash in /usr/local/lib/python3.11/dist-packages (3.1.1)
Requirement already satisfied: Flask<3.2,>=1.0.4 in /usr/local/lib/python3.11/dist-packages (from dash) (3.1.1)
Requirement already satisfied: Werkzeug<3.2 in /usr/local/lib/python3.11/dist-packages (from dash) (3.1.3)
Requirement already satisfied: plotly>=5.0.0 in /usr/local/lib/python3.11/dist-packages (from dash) (5.24.1)
Requirement already satisfied: importlib-metadata in /usr/local/lib/python3.11/dist-packages (from dash) (8.7.0)
Requirement already satisfied: typing-extensions>=4.1.1 in /usr/local/lib/python3.11/dist-packages (from dash) (4.14.0)
Requirement already satisfied: requests in /usr/local/lib/python3.11/dist-packages (from dash) (2.32.3)
Requirement already satisfied: retrying in /usr/local/lib/python3.11/dist-packages (from dash) (1.4.0)
Requirement already satisfied: nest-asyncio in /usr/local/lib/python3.11/dist-packages (from dash) (1.6.0)
Requirement already satisfied: setuptools in /usr/local/lib/python3.11/dist-packages (from dash) (75.2.0)
Requirement already satisfied: blinker>=1.9.0 in /usr/local/lib/python3.11/dist-packages (from Flask<3.2,>=1.0.4->dash) (1.9.0)
Requirement already satisfied: click>=8.1.3 in /usr/local/lib/python3.11/dist-packages (from Flask<3.2,>=1.0.4->dash) (8.2.1)
Requirement already satisfied: itsdangerous>=2.2.0 in /usr/local/lib/python3.11/dist-packages (from Flask<3.2,>=1.0.4->dash) (2.2.0)
Requirement already satisfied: Jinja2>=3.1.2 in /usr/local/lib/python3.11/dist-packages (from Flask<3.2,>=1.0.4->dash) (3.1.6)
Requirement already satisfied: MarkupSafe>=2.1.1 in /usr/local/lib/python3.11/dist-packages (from Flask<3.2,>=1.0.4->dash) (3.0.2)
Requirement already satisfied: tenacity>=6.2.0 in /usr/local/lib/python3.11/dist-packages (from plotly>=5.0.0->dash) (8.5.0)
Requirement already satisfied: packaging in /usr/local/lib/python3.11/dist-packages (from plotly>=5.0.0->dash) (24.2)
Requirement already satisfied: zipp>=3.20 in /usr/local/lib/python3.11/dist-packages (from importlib-metadata->dash) (3.23.0)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.11/dist-packages (from requests->dash) (3.4.2)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.11/dist-packages (from requests->dash) (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.11/dist-packages (from requests->dash) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.11/dist-packages (from requests->dash) (2025.6.15)
```

```
[14] !pip install plotly
Requirement already satisfied: plotly in /usr/local/lib/python3.11/dist-packages (5.24.1)
Requirement already satisfied: tenacity>=6.2.0 in /usr/local/lib/python3.11/dist-packages (from plotly) (8.5.0)
Requirement already satisfied: packaging in /usr/local/lib/python3.11/dist-packages (from plotly) (24.2)
```

```
pip install dash plotly pandas requests
Requirement already satisfied: dash in /usr/local/lib/python3.11/dist-packages (3.1.1)
Requirement already satisfied: plotly in /usr/local/lib/python3.11/dist-packages (5.24.1)
Requirement already satisfied: pandas in /usr/local/lib/python3.11/dist-packages (2.2.2)
Requirement already satisfied: requests in /usr/local/lib/python3.11/dist-packages (2.32.3)
Requirement already satisfied: Flask<3.2,>=1.0.4 in /usr/local/lib/python3.11/dist-packages (from dash) (3.1.1)
Requirement already satisfied: Werkzeug<3.2 in /usr/local/lib/python3.11/dist-packages (from dash) (3.1.3)
Requirement already satisfied: importlib-metadata in /usr/local/lib/python3.11/dist-packages (from dash) (8.7.0)
Requirement already satisfied: typing-extensions>=4.1.1 in /usr/local/lib/python3.11/dist-packages (from dash) (4.14.0)
Requirement already satisfied: retrying in /usr/local/lib/python3.11/dist-packages (from dash) (1.4.0)
Requirement already satisfied: nest-asyncio in /usr/local/lib/python3.11/dist-packages (from dash) (1.6.0)
Requirement already satisfied: setuptools in /usr/local/lib/python3.11/dist-packages (from dash) (75.2.0)
Requirement already satisfied: tenacity>=6.2.0 in /usr/local/lib/python3.11/dist-packages (from plotly) (8.5.0)
Requirement already satisfied: packaging in /usr/local/lib/python3.11/dist-packages (from plotly) (24.2)
Requirement already satisfied: numpy>=1.23.2 in /usr/local/lib/python3.11/dist-packages (from pandas) (2.0.2)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.11/dist-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.11/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.11/dist-packages (from pandas) (2025.2)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.11/dist-packages (from requests) (3.4.2)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.11/dist-packages (from requests) (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.11/dist-packages (from requests) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.11/dist-packages (from requests) (2025.6.15)
Requirement already satisfied: blinker>=1.9.0 in /usr/local/lib/python3.11/dist-packages (from Flask<3.2,>=1.0.4->dash) (1.9.0)
Requirement already satisfied: click>=8.1.3 in /usr/local/lib/python3.11/dist-packages (from Flask<3.2,>=1.0.4->dash) (8.2.1)
Requirement already satisfied: itsdangerous>=2.2.0 in /usr/local/lib/python3.11/dist-packages (from Flask<3.2,>=1.0.4->dash) (2.2.0)
Requirement already satisfied: Jinja2>=3.1.2 in /usr/local/lib/python3.11/dist-packages (from Flask<3.2,>=1.0.4->dash) (3.1.6)
Requirement already satisfied: MarkupSafe>=2.1.1 in /usr/local/lib/python3.11/dist-packages (from Flask<3.2,>=1.0.4->dash) (3.0.2)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.11/dist-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
Requirement already satisfied: zipp>=3.20 in /usr/local/lib/python3.11/dist-packages (from importlib-metadata->dash) (3.23.0)
```

RAW DATA

```
import requests

API_KEY = 'c4dd2f257c9c30f02900eb1e04373c35'
BASE_URL = 'https://api.themoviedb.org/3'

def fetch_movies(pages=5):
    movies = []
    for page in range(1, pages + 1):
        r = requests.get(f"{BASE_URL}/movie/popular", params={
            'api_key': API_KEY,
            'language': 'en-US',
            'page': page
        })
        if r.ok:
            movies.extend(r.json().get('results', []))
        else:
            print(f" Error page {page}: {r.status_code}")

    print(f"\n Total movies: {len(movies)}\n")

    for i, movie in enumerate(movies, start=1):
        title = movie.get('title')
        popularity = movie.get('popularity')
        vote_average = movie.get('vote_average')
        release_date = movie.get('release_date')
        print(f"{i}. {title} | Popularity: {popularity} | Vote Avg: {vote_average} | Release Date: {release_date}")

if __name__ == '__main__':
    fetch_movies()
```

```
Total movies: 100

1. Ballerina | Popularity: 763.892 | Vote Avg: 7.3 | Release Date: 2025-06-04
2. The Old Guard 2 | Popularity: 748.939 | Vote Avg: 6.165 | Release Date: 2025-07-01
3. Thunderbolts* | Popularity: 602.2079 | Vote Avg: 7.4 | Release Date: 2025-04-30
4. Heads of State | Popularity: 599.1112 | Vote Avg: 6.969 | Release Date: 2025-07-02
5. Ice Road: Vengeance | Popularity: 575.9321 | Vote Avg: 7.0 | Release Date: 2025-06-27
6. Jurassic World Rebirth | Popularity: 419.6887 | Vote Avg: 6.4 | Release Date: 2025-07-01
7. Final Destination Bloodlines | Popularity: 293.9397 | Vote Avg: 7.2 | Release Date: 2025-05-14
8. Bring Her Back | Popularity: 280.139 | Vote Avg: 7.3 | Release Date: 2025-05-28
9. Lilo & Stitch | Popularity: 243.9497 | Vote Avg: 7.1 | Release Date: 2025-05-17
10. KPop Demon Hunters | Popularity: 226.7064 | Vote Avg: 8.611 | Release Date: 2025-06-20
11. Dora and the Search for Sol Dorado | Popularity: 219.2418 | Vote Avg: 6.9 | Release Date: 2025-07-02
12. Muromachi Burai | Popularity: 218.5835 | Vote Avg: 9.1 | Release Date: 2025-01-17
13. The Ritual | Popularity: 209.0351 | Vote Avg: 6.0 | Release Date: 2025-04-18
14. How to Train Your Dragon | Popularity: 200.2922 | Vote Avg: 7.9 | Release Date: 2025-06-06
15. F1 | Popularity: 195.9482 | Vote Avg: 7.679 | Release Date: 2025-06-25
16. Sinners | Popularity: 134.8552 | Vote Avg: 7.552 | Release Date: 2025-04-16
17. Long Distance | Popularity: 133.8274 | Vote Avg: 6.367 | Release Date: 2024-07-12
18. The Twisters | Popularity: 128.5091 | Vote Avg: 5.858 | Release Date: 2024-06-28
19. xXx | Popularity: 126.0458 | Vote Avg: 5.945 | Release Date: 2002-08-09
20. 28 Years Later | Popularity: 120.4782 | Vote Avg: 7.2 | Release Date: 2025-06-18
21. A Minecraft Movie | Popularity: 119.4086 | Vote Avg: 6.459 | Release Date: 2025-03-31
22. Superman | Popularity: 112.9104 | Vote Avg: 6.8 | Release Date: 2025-07-09
23. Mission: Impossible - The Final Reckoning | Popularity: 111.2379 | Vote Avg: 7.2 | Release Date: 2025-05-17
24. Crazy Lizard | Popularity: 110.5567 | Vote Avg: 6.042 | Release Date: 2024-03-27
25. First Shift | Popularity: 109.1247 | Vote Avg: 5.667 | Release Date: 2024-08-30
26. Diablo | Popularity: 102.8715 | Vote Avg: 7.186 | Release Date: 2025-06-13
27. Squid Game: Making Season 2 | Popularity: 101.9965 | Vote Avg: 8.498 | Release Date: 2025-01-02
28. The Amateur | Popularity: 101.8252 | Vote Avg: 7.004 | Release Date: 2025-04-09
29. M3GAN 2.0 | Popularity: 96.4968 | Vote Avg: 7.2 | Release Date: 2025-06-25
30. STRAW | Popularity: 87.6636 | Vote Avg: 7.942 | Release Date: 2025-06-05
31. Laila | Popularity: 85.5799 | Vote Avg: 5.8 | Release Date: 2025-02-14
32. Jokōsei torio: seikan shiken | Popularity: 85.1408 | Vote Avg: 7.3 | Release Date: 1977-02-23
33. Candle in the Tomb: The Worm Valley | Popularity: 81.3896 | Vote Avg: 6.818 | Release Date: 2023-09-22
34. Predator: Killer of Killers | Popularity: 79.8428 | Vote Avg: 7.961 | Release Date: 2025-06-05
35. Clown in a Cornfield | Popularity: 76.3579 | Vote Avg: 6.3 | Release Date: 2025-05-08
36. The Old Guard | Popularity: 76.1433 | Vote Avg: 7.139 | Release Date: 2020-07-09
37. A Working Man | Popularity: 75.391 | Vote Avg: 6.689 | Release Date: 2025-03-26
38. The Accountant² | Popularity: 74.6998 | Vote Avg: 7.2 | Release Date: 2025-04-23
39. Jurassic World Dominion | Popularity: 71.6862 | Vote Avg: 6.656 | Release Date: 2022-06-01
40. Squid Game in Conversation | Popularity: 70.3175 | Vote Avg: 8.8 | Release Date: 2025-06-26
41. Moana 2 | Popularity: 64.8332 | Vote Avg: 7.071 | Release Date: 2024-11-21
42. K.O. | Popularity: 60.438 | Vote Avg: 6.574 | Release Date: 2025-06-05
43. Fierce Town | Popularity: 53.8201 | Vote Avg: 7.417 | Release Date: 2025-01-30
44. Captain America: Brave New World | Popularity: 58.353 | Vote Avg: 6.0 | Release Date: 2025-02-12
```

DATA FRAME

```
[17] import requests
import pandas as pd

API_KEY = 'c4dd2f257c9c30f02900e1e04373c35'
BASE_URL = 'https://api.themoviedb.org/3'

def fetch_movie_data():
    movies = []
    for page in range(1, 6):
        response = requests.get(f"{BASE_URL}/movie/popular", params={
            'api_key': API_KEY,
            'language': 'en-US',
            'page': page
        })
        if response.status_code == 200:
            data = response.json().get('results', [])
            movies.extend(data)
        else:
            print(f"Gagal mengambil data dari halaman {page}: {response.status_code}")

    df = pd.DataFrame(movies)

    df['release_date'] = pd.to_datetime(df['release_date'], errors='coerce')
    df['year'] = df['release_date'].dt.year
    df['popularity'] = pd.to_numeric(df['popularity'], errors='coerce')

    return df

if __name__ == '__main__':
    df = fetch_movie_data()

    columns_to_show = ['title', 'popularity', 'vote_average', 'release_date', 'year']

    pd.set_option('display.max_rows', None)
    pd.set_option('display.max_columns', None)
    pd.set_option('display.width', None)
    pd.set_option('display.colheader_justify', 'center')

    print("\n Daftar Film Populer dari TMDb:\n")
    print(df[columns_to_show].head(100).to_string(index=False))
```

Daftar Film Populer dari TMDb:

title	popularity	vote_average	release_date	year
Ballerina	763.8920	7.300	2025-06-04	2025
The Old Guard 2	748.9390	6.165	2025-07-01	2025
Thunderbolts*	602.2079	7.400	2025-04-30	2025
Heads of State	599.1112	6.969	2025-07-02	2025
Ice Road: Vengeance	575.9321	7.000	2025-06-27	2025
Jurassic World Rebirth	419.6887	6.400	2025-07-01	2025
Final Destination Bloodlines	293.9397	7.200	2025-05-14	2025
Bring Her Back	280.1390	7.300	2025-05-28	2025
Lilo & Stitch	243.9497	7.100	2025-05-17	2025
KPop Demon Hunters	226.7064	8.611	2025-06-20	2025
Dora and the Search for Sol Dorado	219.2418	6.900	2025-07-02	2025
Muromachi Burai	218.5835	9.100	2025-01-17	2025
The Ritual	209.0351	6.000	2025-04-18	2025
How to Train Your Dragon	200.2922	7.900	2025-06-06	2025
F1	195.9482	7.679	2025-06-25	2025
Sinners	134.8552	7.552	2025-04-16	2025
Long Distance	133.8274	6.367	2024-07-12	2024
The Twisters	128.5091	5.858	2024-06-28	2024
xXx	126.0458	5.945	2002-08-09	2002
28 Years Later	120.4782	7.200	2025-06-18	2025
A Minecraft Movie	119.4086	6.459	2025-03-31	2025
Superman	112.9104	6.800	2025-07-09	2025
Mission: Impossible - The Final Reckoning	111.2379	7.200	2025-05-17	2025
Crazy Lizard	110.5567	6.042	2024-03-27	2024
First Shift	109.1247	5.667	2024-08-30	2024
Diablo	102.8715	7.186	2025-06-13	2025
Squid Game: Making Season 2	101.9965	8.498	2025-01-02	2025
The Amateur	101.8252	7.004	2025-04-09	2025
M3GAN 2.0	96.4968	7.200	2025-06-25	2025
STRAW	87.6636	7.942	2025-06-05	2025
Laila	85.5799	5.800	2025-02-14	2025
Jokōsei torio: seikan shiken	85.1408	7.300	1977-02-23	1977
Candle in the Tomb: The Worm Valley	81.3896	6.818	2023-09-22	2023
Predator: Killer of Killers	79.8428	7.961	2025-06-05	2025
Clown in a Cornfield	76.3579	6.300	2025-05-08	2025
The Old Guard	76.1433	7.139	2020-07-09	2020
A Working Man	75.3910	6.689	2025-03-26	2025
The Accountant²	74.6998	7.200	2025-04-23	2025
Jurassic World Dominion	71.6862	6.656	2022-06-01	2022
Squid Game in Conversation	70.3175	8.800	2025-06-26	2025
Moana 2	64.8332	7.071	2024-11-21	2024
K.O.	60.4380	6.574	2025-06-05	2025
Fierce Town	53.8201	7.417	2025-01-30	2025
Captain America: Brave New World	58.3530	6.000	2025-02-12	2025
My Fault	60.7491	7.798	2023-06-08	2023

CSV

```
if __name__ == '__main__':
    df = fetch_movie_data()

    columns_to_show = ['title', 'popularity', 'vote_average', 'release_date', 'year']

    pd.set_option('display.max_rows', None)
    pd.set_option('display.max_columns', None)
    pd.set_option('display.width', None)
    pd.set_option('display.colheader_justify', 'center')

    print("\n Daftar Film Populer dari TMDb:\n")
    print(df[columns_to_show].head(100).to_string(index=False))

    df[columns_to_show].to_csv('tmdb_movies.csv', index=False)
    print("\nData berhasil disimpan ke file CSV: tmdb_movies.csv")
```

Daftar Film Populer dari TMDb:

title	popularity	vote_average	release_date	year
Ballerina	763.8920	7.300	2025-06-04	2025
The Old Guard 2	748.9390	6.165	2025-07-01	2025
Thunderbolts*	602.2079	7.400	2025-04-30	2025
Heads of State	599.1112	6.969	2025-07-02	2025
Ice Road: Vengeance	575.9321	7.000	2025-06-27	2025
Jurassic World Rebirth	419.6887	6.400	2025-07-01	2025
Final Destination Bloodlines	293.9397	7.200	2025-05-14	2025
Bring Her Back	280.1390	7.300	2025-05-28	2025
Lilo & Stitch	243.9497	7.100	2025-05-17	2025
KPop Demon Hunters	226.7064	8.611	2025-06-20	2025
Dora and the Search for Sol Dorado	219.2418	6.900	2025-07-02	2025
Muromachi Burai	218.5835	9.100	2025-01-17	2025
The Ritual	209.0351	6.000	2025-04-18	2025
How to Train Your Dragon	200.2922	7.900	2025-06-06	2025
F1	195.9482	7.679	2025-06-25	2025
Sinners	134.8552	7.552	2025-04-16	2025
Long Distance	133.8274	6.367	2024-07-12	2024
The Twisters	128.5091	5.858	2024-06-28	2024
xXx	126.0458	5.945	2002-08-09	2002
28 Years Later	120.4782	7.200	2025-06-18	2025
A Minecraft Movie	119.4086	6.459	2025-03-31	2025
Superman	112.9104	6.800	2025-07-09	2025
Mission: Impossible - The Final Reckoning	111.2379	7.200	2025-05-17	2025
Crazy Lizard	110.5567	6.042	2024-03-27	2024
First Shift	109.1247	5.667	2024-08-30	2024
Diablo	102.8715	7.186	2025-06-13	2025
Squid Game: Making Season 2	101.9965	8.498	2025-01-02	2025
The Amateur	101.8252	7.004	2025-04-09	2025

AutoSaveOfftmdb_movies.csv

Search

FileHomeWPS PDFInsertDrawPage LayoutFormulasDataReviewViewHelp

Clipboard

Font

Alignment

Calibri11A

B

I

U

<

PREPROCESS

```
import requests
import pandas as pd

API_KEY = 'c4dd2f257c9c30f02900eb1e04373c35'
BASE_URL = 'https://api.themoviedb.org/3'
CSV_FILE = 'tmdb_movies.csv'

def process_fetch_data(pages=5):
    movies = []
    for page in range(1, pages + 1):
        response = requests.get(f"{BASE_URL}/movie/popular", params={
            'api_key': API_KEY,
            'language': 'en-US',
            'page': page
        })
        if response.ok:
            movies.extend(response.json().get('results', []))
        else:
            print(f"Gagal mengambil data halaman {page}: {response.status_code}")
    return movies

def process_preprocessing(movies):
    df = pd.DataFrame(movies)
    df['release_date'] = pd.to_datetime(df['release_date'], errors='coerce')
    df['year'] = df['release_date'].dt.year
    df['popularity'] = pd.to_numeric(df['popularity'], errors='coerce')
    df = df.drop_duplicates(subset='title')
    return df

def process_display_and_save(df):
    columns = ['title', 'popularity', 'vote_average', 'release_date', 'year']
    pd.set_option('display.max_rows', None)
    print("\nData TMDb:\n")
    print(df[columns].head(100).to_string(index=False))

    df[columns].to_csv(CSV_FILE, index=False)
    print(f"\nData disimpan ke {CSV_FILE}")

if __name__ == '__main__':
    movies = process_fetch_data()
    df = process_preprocessing(movies)
    process_display_and_save(df)
```

Data TMDb:

title	popularity	vote_average	release_date	year
Ballerina	763.8920	7.300	2025-06-04	2025
The Old Guard 2	748.9390	6.165	2025-07-01	2025
Thunderbolts*	602.2079	7.400	2025-04-30	2025
Heads of State	599.1112	6.969	2025-07-02	2025
Ice Road: Vengeance	575.9321	7.000	2025-06-27	2025
Jurassic World Rebirth	419.6887	6.400	2025-07-01	2025
Final Destination Bloodlines	293.9397	7.200	2025-05-14	2025
Bring Her Back	280.1390	7.300	2025-05-28	2025
Lilo & Stitch	243.9497	7.100	2025-05-17	2025
KPop Demon Hunters	226.7064	8.611	2025-06-20	2025
Dora and the Search for Sol Dorado	219.2418	6.900	2025-07-02	2025
Muromachi Burai	218.5835	9.100	2025-01-17	2025
The Ritual	209.0351	6.000	2025-04-18	2025
How to Train Your Dragon	200.2922	7.900	2025-06-06	2025
F1	195.9482	7.679	2025-06-25	2025
Sinners	134.8552	7.552	2025-04-16	2025
Long Distance	133.8274	6.367	2024-07-12	2024
The Twisters	128.5091	5.858	2024-06-28	2024
xxX	126.0458	5.945	2002-08-09	2002
28 Years Later	120.4782	7.200	2025-06-18	2025
A Minecraft Movie	119.4086	6.459	2025-03-31	2025
Superman	112.9104	6.800	2025-07-09	2025
Mission: Impossible - The Final Reckoning	111.2379	7.200	2025-05-17	2025
Crazy Lizard	110.5567	6.042	2024-03-27	2024
First Shift	109.1247	5.667	2024-08-30	2024
Diablo	102.8715	7.186	2025-06-13	2025
Squid Game: Making Season 2	101.9965	8.498	2025-01-02	2025
The Amateur	101.8252	7.004	2025-04-09	2025
M3GAN 2.0	96.4968	7.200	2025-06-25	2025
STRAW	87.6636	7.942	2025-06-05	2025
Laila	85.5799	5.800	2025-02-14	2025
Jokōsei torio: seikan shiken	85.1408	7.300	1977-02-23	1977
Candle in the Tomb: The Worm Valley	81.3896	6.818	2023-09-22	2023
Predator: Killer of Killers	79.8428	7.961	2025-06-05	2025
Clown in a Cornfield	76.3579	6.300	2025-05-08	2025
The Old Guard	76.1433	7.139	2020-07-09	2020
A Working Man	75.3910	6.689	2025-03-26	2025
The Accountant²	74.6998	7.200	2025-04-23	2025
Jurassic World Dominion	71.6862	6.656	2022-06-01	2022
Squid Game in Conversation	70.3175	8.800	2025-06-26	2025
Moana 2	64.8332	7.071	2024-11-21	2024
K.O.	60.4380	6.574	2025-06-05	2025
Fierce Town	53.8201	7.417	2025-01-30	2025
Captain America: Brave New World	58.3530	6.000	2025-02-12	2025

VISUALISASI

```
from dash import Dash, html, dash_table, dcc, callback, Output, Input
import requests
import pandas as pd
import plotly.express as px

API_KEY = 'c4dd2f257c9c30f02900eb1e04373c35'
BASE_URL = 'https://api.themoviedb.org/3'

def get_movie_data():
    movies = []
    for page in range(1, 6):
        response = requests.get(f"{BASE_URL}/movie/popular", params={
            'api_key': API_KEY,
            'language': 'en-US',
            'page': page
        })
        if response.status_code == 200:
            data = response.json().get('results', [])
            movies.extend(data)
        else:
            print(f"Error page {page}: {response.status_code}")

    df = pd.DataFrame(movies)
    df['release_date'] = pd.to_datetime(df['release_date'], errors='coerce')
    df['year'] = df['release_date'].dt.year
    df['popularity'] = pd.to_numeric(df['popularity'], errors='coerce')
    df['vote_average'] = pd.to_numeric(df['vote_average'], errors='coerce')
    df = df[['title', 'popularity', 'vote_average', 'year']]
    return df

df = get_movie_data()

app = Dash(__name__)
app.title = "TMDb Movie Dashboard"

app.layout = html.Div([
    html.H1(
        "TMDb Movie Dashboard",
        style={'textAlign': 'center', 'color': '#333'}
    ),
    html.Hr(),
    html.Div([
        html.Label("Pilih Data:", style={'fontWeight': 'bold'}),
        dcc.RadioItems(
            options=[
                {'label': 'Popularity', 'value': 'popularity'},
                {'label': 'Vote Average', 'value': 'vote_average'}
            ],

```

```
        value='popularity',
        id='radio-metric',
        inline=True,
        style={'padding': '10px'}
    ),
], style={'marginBottom': '20px'}),

html.Div([
    html.Div([
        dash_table.DataTable(
            id='table-tmdb',
            page_size=10,
            style_table={'overflowX': 'auto'},
            style_cell={'textAlign': 'left', 'padding': '5px'},
            style_header={'backgroundColor': '#0074D9', 'color': 'white', 'fontWeight': 'bold'}
        )
    ], style={'width': '48%', 'display': 'inline-block', 'verticalAlign': 'top'}),

    html.Div([
        dcc.Graph(id='graph-tmdb', figure={})
    ], style={'width': '50%', 'display': 'inline-block', 'paddingLeft': '20px'})
]), style={'padding': '40px', 'fontFamily': 'Arial'})

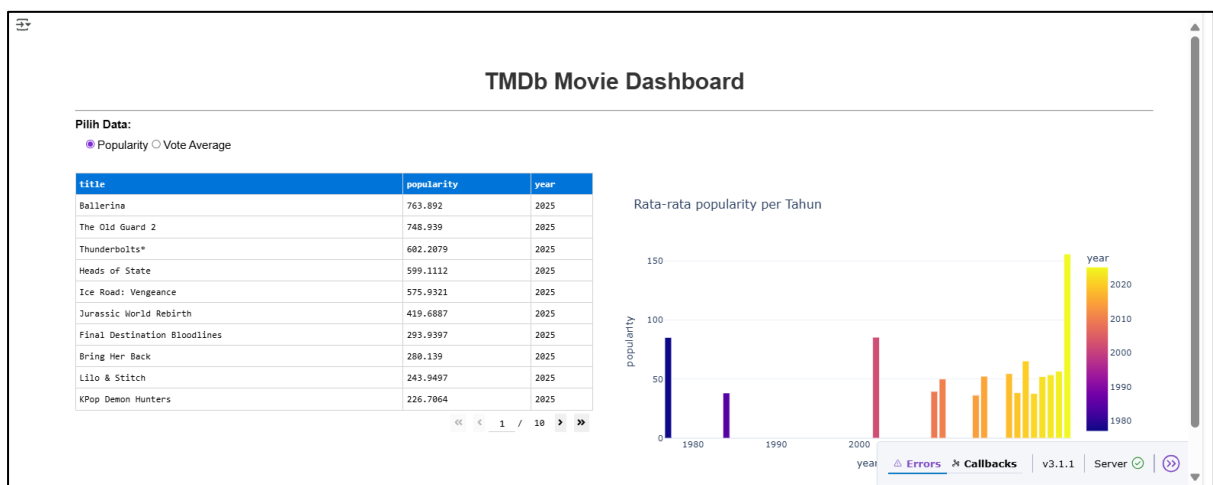
@callback(
    Output('table-tmdb', 'data'),
    Output('table-tmdb', 'columns'),
    Output('graph-tmdb', 'figure'),
    Input('radio-metric', 'value')
)
def update_dashboard(metric):
    if metric == 'popularity':
        columns = ['title', 'popularity', 'year']
    else:
        columns = ['title', 'vote_average', 'year']

    dff = df.dropna(subset=['year', metric])
    data = dff[columns].to_dict('records')
    cols = [{"name": i, "id": i} for i in columns]

    grouped = dff.groupby('year')[metric].mean().reset_index()
    fig = px.bar(
        grouped, x='year', y=metric,
        title=f"Rata-rata {metric} per Tahun",
        color='year',
        template='plotly_white'
    )
    fig.update_layout(transition_duration=500)

    return data, cols, fig

if __name__ == '__main__':
    app.run(debug=True)
```



TMDb Movie Dashboard

Pilih Data:
☐ Popularity ☒ Vote Average

title	vote_average	year
Ballerina	7.357	2025
The Old Guard 2	6.152	2025
Heads of State	7.002	2025
Ice Road: Vengeance	7.013	2025
Thunderbolts*	7.4	2025
Jurassic World Rebirth	6.427	2025
Karate Kid: Legends	6.941	2025
Final Destination Bloodlines	7.216	2025
Bring Her Back	7.321	2025
Lilo & Stitch	7.136	2025

Rata-rata vote_average per Tahun



HASIL DAN PEMBAHASAN

Hasil dari pengerjaan ini berupa:

- Data film populer sebanyak lebih dari 100 judul berhasil diambil dari TMDb melalui crawling lima halaman API.
- Data mentah telah disimpan dalam file CSV sehingga memudahkan replikasi analisis di kemudian hari.
- Setelah dilakukan preprocessing, dataset menjadi lebih bersih dan siap dianalisis.
- Dashboard interaktif menampilkan data dalam bentuk tabel yang dapat difilter berdasarkan metrik popularitas atau rata-rata vote. Grafik batang juga memperlihatkan tren rata-rata metrik yang dipilih berdasarkan tahun rilis film.

Hasil visualisasi menunjukkan bahwa film-film populer cenderung dirilis dalam rentang waktu tertentu, dan popularitas film dapat bervariasi setiap tahunnya. Dari sini dapat disimpulkan bahwa analisis data film dengan pipeline sederhana ini mampu memberikan wawasan awal tentang tren perilaku penonton dan industri film secara umum.

KESIMPULAN

Berdasarkan hasil pengerjaan tugas Big Data Fundamental dengan studi kasus crawling dan visualisasi data film dari TMDb, dapat disimpulkan bahwa proses pengolahan data memerlukan alur kerja yang terstruktur mulai dari pengambilan data (data ingestion), penyimpanan (storage), pembersihan dan transformasi data (preprocessing), hingga penyajian hasil dalam bentuk visualisasi yang informatif dan interaktif. Seluruh tahapan ini saling terkait dan menjadi dasar dari praktik Big Data Pipeline yang lebih kompleks.

Melalui proses ini, mahasiswa dapat memahami bagaimana data mentah yang diambil langsung dari API publik sering kali tidak dapat digunakan secara langsung, melainkan harus melewati beberapa tahap pembersihan untuk memastikan konsistensi dan akurasi. Dengan memanfaatkan pustaka Python seperti requests, pandas, dan plotly dash, mahasiswa belajar menerapkan teknologi open source untuk membangun dashboard sederhana yang mampu menyajikan insight secara real-time.

Dari visualisasi yang dihasilkan, dapat dilihat bagaimana tren popularitas dan rating film bervariasi dari tahun ke tahun. Hasil ini menunjukkan bahwa data film dapat memberikan informasi penting mengenai pola perilaku penonton, popularitas genre tertentu, hingga potensi pasar bagi pihak industri. Hal ini membuktikan bahwa kemampuan membaca pola data secara visual sangat membantu proses pengambilan keputusan berbasis data (data-driven decision making).

Selain itu, tugas ini juga membekali mahasiswa dengan pemahaman praktis tentang bagaimana API crawling dapat diterapkan untuk kebutuhan data mining di dunia nyata.

Mahasiswa juga dilatih untuk berpikir kritis dalam merancang pipeline sederhana yang dapat dioptimalkan untuk skala data yang lebih besar di masa depan.

Secara keseluruhan, pelaksanaan tugas ini tidak hanya memberikan pengalaman teknis, tetapi juga meningkatkan kesadaran akan pentingnya literasi data dalam menjawab tantangan dunia industri yang semakin bergantung pada teknologi dan data besar. Harapannya, keterampilan dasar ini dapat menjadi pondasi bagi pengembangan proyek-proyek lanjutan yang lebih kompleks di bidang Big Data Analytics dan Business Intelligence.

LAMPIRAN

Link semua file project

https://drive.google.com/drive/folders/1p0504NYiGrel_TlwwX47cQW71shke8Tg?usp=sharing